Qwen

From Ad-Hoc Thresholds to Verified Formulas: A Contract-First Framework for Systemic Timing in Horror$Placeristics

Strengthening the Spine with Derived Timing Signals

The foundational architectural principle of Horror$Place is the "Invariant-Driven Spine & Metric-Aware Contracts" doctrine, which treats game design parameters not as mutable variables but as formal, verifiable contracts governing runtime behavior [16] [65]. The invariant spine—comprising core values such as CIC, DET, UEC, and ARR—functions as an immutable mathematical surface, or the "physics" of player experience, ensuring every mechanic, seed, and engine behavior can be audited against a common ground truth [23] [44]. This research focuses on extending this robust framework to govern systemic timing mechanics, which control emergent phenomena like grace periods, punishment escalation, and adaptive difficulty. The central challenge is to introduce new levers for controlling these timing behaviors without polluting the core geo-historical invariant spine, thereby preserving the system's integrity and engine-agnostic promise.

A critical decision in this extension is the nature of the new timing signals. Initial proposals included several candidate invariants: TMI (Timing Mercy Index), RPI (Repetition Punishment Index), ADI (Adaptive Damage Index), and STI (Stress Trajectory Index) [13] [14]. However, subsequent analysis and conversation clarify that these signals should not be implemented as first-class, global invariants alongside established constants like CIC or MDI [74]. Such an approach would risk cluttering the core invariant set with transient, session-scoped, or mechanic-specific state, undermining the clarity and stability of the foundational spine. Instead, the consensus is to treat these signals as **derived, session-scoped concepts**, expressed explicitly within pacing/timing contracts and telemetry schemas, mirroring the treatment of other derived metrics like SPR (Systemic Punishment Rate) and RWF (Respawn Weight Factor) [74]. This strategy preserves the distinction between fundamental, persistent properties of the game world and dynamic, context-dependent behavioral parameters. If these signals are ever considered for inclusion in the main invariant spine, they must be clearly marked as "derived/session-scoped," and their handling in validation tools like `validate-`

`invariants.rs` must be distinct from primary invariants, complete with separate error bands and diagnostic codes [61].

This refined strategy necessitates a clear separation of concerns. Core invariants remain tied to the `invariants_v1.json` schema in the `Horror.Place` repository, defining the fundamental rules of the game world [60]. In contrast, the new timing signals and their associated parameters are managed through a new, dedicated schema, `mechanic_contract_timing_params_v1.json`, located in the `HororPlace-Constellation-Contracts` repository [5]. This structure ensures that any changes to systemic timing logic are contained within a specific, well-defined contract layer, making them easier to reason about, validate, and tune independently of the core game physics. For instance, TMI and RPI can be modeled as functionally derivable from core invariants, metrics, and telemetry history rather than being stored as primitive fields themselves [70]. TMI could be a sigmoidal transform of a moving average of DET combined with a negative function of recent death frustration, while RPI could be a logistic function of the local slope of death counts versus attempts within a given pattern. These derived signals would then be computed within Lua helpers based on the player's history and passed as inputs to the timing formulas, ensuring that the core invariant set remains stable and focused on long-term, geo-historical states [51].

The implementation of this approach requires careful consideration of schema design and tooling integration. The `mechanic_contract_timing_params_v1.json` schema defines the precise structure and constraints for all coefficients and parameters used in the systemic timing formulas [69]. It establishes ranges for weights (e.g., `w1`, `a1`), thresholds, and normalization bounds (e.g., `det_min`, `det_max`, `arr_floor`), preventing invalid configurations from being deployed [30]. Furthermore, this schema must incorporate hooks for "narrowing," a crucial validation rule that ensures child contracts (e.g., a specific event or sequence seed) do not widen the parameter bands defined by their parent envelopes (policy, region, style) [26]. This prevents unintended escalation of timing mechanics, where a highly constrained region might inadvertently spawn a mechanic with overly aggressive parameters. By enforcing interval inclusion at the contract level, the system maintains its hierarchical integrity, ensuring that local behavior always adheres to broader design goals [50]. The table below outlines the proposed structure for these timing parameters, demonstrating how they provide the necessary tunable knobs for the formula-driven helpers while remaining strictly within the contract-first doctrine.

Parameter Group	Key Coefficient(s)	Description	Typical Range
Coyote Time	`w1`, `w2`	Weights for DET and inverse ARR in the probability calculation. Controls mercy granting sensitivity.	`[0, 10]`
Coyote Time	`threshold`	Sigmoid threshold determining the baseline probability.	Unbounded
Rule of Three	`a1`, `a2`, `a3`, `a4`	Coefficients for DET, UEC, inverse ARR, and death count in the downgrade probability calculation.	`[0, ∞)`
Rule of Three	`bias`	Baseline bias term for the downgrade probability.	Unbounded
Adaptive Damage	`m_base`, `alpha`, `beta`	Base damage multiplier and sensitivity coefficients for DET and ARR.	`m_base > 0`, `[-1, 1]`
Adaptive Damage	`m_min`, `m_max`	Hard bounds for the final damage multiplier.	`[0, ∞)`

By adopting this model, Horror$Place can evolve its systemic timing capabilities in a controlled, predictable, and mathematically grounded manner. The invariant-driven spine provides the stable foundation, while the contract-driven parameter schemas offer the flexibility needed to fine-tune emergent player experiences. This approach directly addresses the identified gap in the current architecture by providing the same formal, verifiable status to systemic timing logic as it already has for spectral logic, ensuring that all timing behavior remains explainable in the CIC/DET/LSG/UEC/ARR space [62] [64].

Developing Formula-Driven Lua Helpers with Bounded Properties

A significant vulnerability in the current Horror$Place implementation lies in the use of hardcoded thresholds within Lua scripts for systemic timing decisions, such as granting grace periods or downgrading lethal hits [9] [31]. This practice leads to "code drift," where balancing decisions become opaque, non-reproducible, and difficult to port across different engine implementations, ultimately hindering the project's goal of creating an engine-agnostic system [7]. To address this, the research proposes replacing these ad-hoc thresholds with a suite of formula-driven Lua helpers. This approach transforms timing logic from a series of hard-coded if-statements into a set of analyzable functions, where probabilities and multipliers are direct mathematical outputs of the player's invariant and metric state [16]. This shift enables a more systematic, data-driven process for tuning and validating timing behavior.

The core idea is to replace constructs like `if H.DET(region_id, player_id) > 0.7 then grant_coyote_time()` with a probabilistic function that maps the normalized state of key metrics to an outcome. The provided materials sketch concrete mathematical formulas for three primary systemic timing mechanics: Coyote Time, Rule of Three, and Adaptive Damage. These formulas are designed to be simple, analyzable, and to satisfy important mathematical properties that guarantee predictable behavior. They typically employ a sigmoid function, $\sigma(x)=\frac{1}{1+e^{-x}}$, to smoothly map an unbounded input to a bounded output, such as a probability in the range [0, 1] [3] [4]. This ensures that small changes in player state lead to smooth changes in mechanic behavior, avoiding the jarring, step-like transitions caused by discrete thresholds.

For **Coyote Time**, the probability of granting a grace window is calculated as a function of normalized Dread Exposure (DET) and Adaptation Response Rate (ARR). The formula is designed to make mercy more likely when the player is under high stress (high DET) and less adapted (low ARR).

p_{\text{coyote}} = \sigma\left(w_1 \cdot \frac{\text{DET} - \text{DET}<em _text_ran

In this equation, w_1 and w_2 are tunable weights, and T is a threshold parameter, all sourced from a mechanic contract. The normalization terms scale DET and ARR to a [0, 1] range based on the contract-specified minimum and maximum values. This formula guarantees that p_{coyote} is always in [0, 1] and is monotonically increasing with respect to DET and decreasing with respect to ARR, aligning with the intended design [69].}}

For the **Rule of Three**, which escalates punishment after repeated failures, the downgrade probability is a function of multiple factors: normalized DET, UEC, ARR, and a death counter. The goal is to increase the likelihood of a near-lethal downgrade when the player is struggling (high DET), has low engagement (low UEC), is poorly adapted (low ARR), and has failed multiple times.

p_{\text{downgrade}} = \sigma\left(-a_1 \cdot \frac{\text{DET}}{\text{DET}<em _tex

Here, a_1 through a_4 and B are contract parameters. The formula is structured to make downgrades more probable under the specified stressful conditions, with the final probability clamped to [0, 1] [73].}}}\right) + a_4 \cdot \frac{\text{deathCount}}{\text{maxAttempts}

Finally, for **Adaptive Damage**, the damage multiplier is scaled based on the player's state. The formula aims to increase damage when the player is under high stress and decrease it as they adapt, creating a dynamic difficulty curve.

m = \text{clip}\left(m_{\text{base}} \cdot \left(1 + \alpha \cdot \frac{\text{DET} - \tex

This formula multiplies a base damage value (m_{base}) by scaling factors derived from normalized DET and ARR. The coefficients α and β determine the sensitivity to dread and adaptation, respectively. The }}}\right), m_{\text{min}}, m_{\text{max}}`clip` function ensures the final multiplier always stays within the contract-defined bounds $[m_{\text{min}}, m_{\text{max}}]$ [59].

To ensure the reliability of these formulas, the research prioritizes establishing basic formal guarantees *before* full-scale implementation. This involves proving key properties once, using a proof assistant like Lean or Coq, to provide machine-checked evidence of correctness [54] [55]. The primary properties to be proven are: 1. **Boundedness:** All helper functions must produce outputs that lie within their specified mathematical domains. For example, both `p_coyote` and `p_downgrade` must be formally proven to always return a value in the range [0, 1], and the `damage_multiplier` must be proven to be in $[m_{\text{min}}, m_{\text{max}}]$ [69]. 2. **Monotonicity:** The relationship between inputs and outputs must be predictable and align with design intent. For instance, one would prove that the downgrade probability $p_{\text{downgrade}}$ is a non-decreasing function of UEC and death count, and a non-increasing function of ARR. Similarly, it should be proven that the coyote time probability p_{coyote} is a non-decreasing function of DET and a non-increasing function of ARR [56].

These proofs create a trusted foundation upon which the Lua implementations can be built. While full formal verification is a future goal, starting with these essential properties provides immediate benefits. It reduces the cognitive load on developers, eliminates entire classes of bugs related to out-of-bounds values or illogical behavior, and makes the timing logic itself auditable. The Lua helpers themselves would be implemented as a single module (e.g., `engine/systemic_timing.lua`) that exposes these functions, sourcing all coefficients from the mechanic contracts rather than containing any hardcoded values [19] [27]. This modular approach, combined with the analytical power of the underlying formulas, represents a significant leap forward in the quality, maintainability, and predictability of Horror$Place's systemic timing mechanics.

Ensuring Cross-Engine API Consistency

A cornerstone of the Horror$Place architecture is its commitment to being engine-agnostic, allowing the same high-level design contracts to drive different game engines like Unreal (C++), Unity (C#), or Bevy (Rust) [7] [57]. This vision is critically dependent on a consistent and well-defined interface between the invariant-aware policy layer (Lua) and the engine-specific execution layer (Rust/C++/C#). Without such a guarantee, subtle differences in implementation—even for seemingly identical functions—can lead to divergent gameplay experiences, effectively breaking the contract-first doctrine and reintroducing the very "code drift" the architecture was designed to prevent [40]. To mitigate this risk, the research proposes the creation and enforcement of a canonical `engine_api_surface_v1.json` schema, which serves as the single source of truth for the `H.*` API that all engine implementations must expose [5].

The `engine_api_surface_v1.json` schema is designed to formalize the contract between the scripting environment and the engine. Its purpose is to eliminate ambiguity and provide a machine-enforceable specification for all functions that Lua scripts can call to query invariant and metric data or to request engine actions. The schema would be structured to define several key functional groups, each with explicit signatures, argument types, and return types. This level of formality is essential for enabling automated validation and ensuring interoperability across platforms. The proposed schema structure includes three main sections: invariant queries, systemic timing helpers, and telemetry hooks [25] [46].

The `invariant_queries` section would standardize the functions that allow the Lua layer to read the current state of the game world. This mirrors the existing narrow `H.*` bridge but elevates it to a formal, documented contract. Functions like `H.CIC`, `H.DET`, `H.UEC`, and `H.ARR` would be defined with precise signatures, such as `(region_id: str, tile_id: str) -> float` for `CIC` and `(region_id: str, player_id: str) -> float` for `DET`. This ensures that every engine implementation provides the exact same query interface, preventing discrepancies where one engine might require a different number or type of arguments [26].

The second, and most critical, part of the schema is the `systemic_timing_helpers` section. This defines the standardized signatures for the formula-driven Lua helpers that implement timing mechanics. Functions like `CoyoteTime_maybe_grant`, `RuleOfThree_maybe_downgrade`, and `AdaptiveDamage_scale` would have their input and output structures formally specified. For example, the signature for `CoyoteTime_maybe_grant` might be defined as `(region_id: str, player_id:`

`str, contract: object) -> GrantResult`, where `GrantResult` is another defined object shape. This ensures that regardless of whether the underlying engine is Rust, C++, or C#, the Lua script sees a consistent interface for requesting timing-related actions. The schema should mandate the H.* query interface and standardize the timing helper signatures without requiring every individual helper as a mandatory primitive [20]. This provides a stable contract while still allowing engines to add local wrappers or variants behind that stable interface, accommodating platform-specific optimizations without altering the core policy logic [26].

The third section, `telemetry_hooks`, would define the interface for logging events back to the central telemetry pipeline. Functions like `log_execution_string` and `log_systemic_timing` would have their payload structures formally defined, ensuring that data emitted by different engines is consistently shaped and can be aggregated for analysis [25]. This consistency is vital for the data-driven refinement loop, where analysis notebooks correlate mechanic decisions with their effects on player metrics [72].

Once defined, the `engine_api_surface_v1.json` schema becomes a powerful enforcement tool. Continuous Integration (CI) workflows can be configured to automatically validate engine implementations against this canonical specification. For a Rust engine, this could involve compiling a test harness that calls the `H.*` stubs and checking that their signatures match the schema. For C++ and C#, similar checks could be performed using language-specific tools or by generating test fixtures from the schema. Any deviation from the canonical API surface would cause the CI build to fail, preventing incompatible changes from being merged into the main codebase [29]. This creates a hard boundary between the invariant-aware policy layer and the engine-specific implementation, fostering true interoperability. The schema acts as a proxy for the formal methods research; the mathematical properties proven about the Lua formulas (monotonicity, boundedness) translate into concrete constraints on the `engine_api_surface_v1.json` schema, which in turn can be checked at compile-time or runtime by the engine adapters [21]. This multi-layered approach—combining a formal schema, automated CI checks, and connections to mathematical proofs—ensures that the promise of an engine-agnostic, invariant-driven architecture is upheld, providing a stable and reliable foundation for the entire Horror$Place ecosystem.

Instrumentation and Empirical Validation via Telemetry

For any complex, emergent system like Horror$Place, theoretical models and mathematical formulas are only as good as their real-world performance. The transition from ad-hoc thresholds to formula-driven systemic timing mechanics necessitates a robust instrumentation and empirical validation strategy. The core of this strategy is a closed-loop feedback system that connects design intent, runtime behavior, and measurable outcomes on player experience metrics like UEC and ARR [6] [45]. This loop relies heavily on detailed telemetry, which captures the causal chain from invariant state to mechanic decision to metric outcome, providing the raw data needed for analysis and refinement [34]. To support this, the research calls for the development of a new, specialized NDJSON schema: `systemic-timing-formula-run.v1`.

The `systemic-timing-run.v1` schema is designed to log every invocation of a systemic timing helper with exhaustive detail. Each record in this telemetry stream should capture the complete context of the decision-making process. This includes the input invariants and metrics at the moment of the call (e.g., `metricsBefore.DET`, `metricsBefore.ARR`, `metricsBefore.UEC`), the specific contract parameters that governed the formula (e.g., `formulaParameters.w1`, `formulaParameters.a2`, `formulaParameters.threshold`), the raw and clamped computed output of the formula (e.g., `computedOutput.rawValue`, `computedOutput.clampedValue`), the final decision made by the helper (e.g., `decisionTaken: "grant_coyote"`), and the resulting change in player metrics (e.g., `outcomeMetrics.UEC_delta`, `outcomeMetrics.ARR_delta`) [35] [73]. By structuring telemetry in this "before/after" format with explicit input/output fields, the data becomes amenable to delta analysis, allowing analysts to precisely measure the impact of a single mechanic decision on the player's state [75].

The implementation of this telemetry strategy involves two key components: instrumenting the Lua helpers and building the data analysis infrastructure. First, every formula-driven helper in the `systemic_timing.lua` module must be modified to emit a `systemic-timing-formula-run.v1` record immediately after computing its result. This logging step must be an integral part of the helper's logic, ensuring that no timing decision is made without being captured in the telemetry stream. Second, this raw telemetry data must be processed and analyzed. This is typically done using aggregation scripts or Jupyter notebooks, such as the `systemic_timing_analysis.ipynb` mentioned in the research documents [74]. These tools would load the NDJSON streams, group events by helper kind and variant, and compute aggregate statistics. For example, an analyst could calculate the mean `UEC_delta` and `ARR_delta` for all instances where

the `CoyoteTime_probability` helper returned a value greater than 0.5 versus those where it returned a value less than 0.5. This allows for a quantitative assessment of whether the timing mechanics are achieving their intended effects.

The ultimate goal of this empirical validation is to move parameter tuning from guesswork to a guided search constrained by both formal properties and observed data [36]. The analysis notebooks serve as the playground for this process. By correlating the computed outputs of the timing formulas with the resulting changes in UEC and ARR, designers can identify effective parameter sets. For instance, the analysis might reveal that a particular combination of weights (`w1`, `w2`) and a specific threshold value produces a statistically significant uplift in UEC without causing the ARR to collapse below its floor, indicating a successful balance between tension and fairness [59]. Conversely, it might show that a certain parameter set leads to excessive punishment, collapsing ARR and frustrating players. In either case, the data provides a clear signal for refinement. Designers can then adjust the coefficients in the relevant mechanic contracts, re-run the simulations or playtests, and observe the new telemetry data to see if the adjustments had the desired effect [38]. This iterative cycle of propose, test, analyze, and refine is the essence of the data-driven approach. The `systemic-timing-formula-run.v1` schema, by providing a rich, structured, and machine-readable record of every timing decision and its consequences, is the linchpin of this entire process, transforming the systemic timing system from a black box into a transparent, analyzable, and empirically validated component of the larger Horror$Place architecture.

Formal Verification and Machine-Checked Proofs

While empirical validation through telemetry provides invaluable data on the practical performance of systemic timing mechanics, it cannot guarantee the absence of subtle logical errors or violations of fundamental constraints. A single anomalous data point might be noise, but a logical flaw in the formula itself could lead to unpredictable and undesirable behavior under a wide range of conditions. To address this gap, the research advocates for a parallel track of formal verification, aiming to supplement empirical testing with mathematical certainty. This involves encoding the timing formulas in a proof assistant like Lean or Coq and constructing machine-checked proofs of their key properties, such as monotonicity and boundedness [54] [55]. This approach provides a higher level of assurance than unit tests alone, as proofs establish properties for the entire domain of possible inputs, not just a finite set of test cases.

The process begins by translating the informal mathematical descriptions of the timing formulas into a formal language within the proof assistant. For example, the `p_coyote` probability function would be defined as a Lean function taking real numbers for DET, ARR, and all the contract-specified parameters (weights, thresholds, min/max values) [56]. The core of the work then involves proving lemmas about this function. One would first prove that the underlying sigmoid function is monotonically increasing. Then, leveraging this fact and the linearity of the expressions for `det_norm` and `arr_norm`, one would prove that the overall `p_coyote` function is monotonically increasing with respect to DET and monotonically decreasing with respect to ARR, assuming positive weights. Similarly, proofs would be constructed to show that the output of `p_coyote` is always clamped to the [0, 1] interval, based on the properties of the sigmoid function and the clamping logic [69].

These formal proofs serve multiple purposes. First, they act as a rigorous sanity check on the mathematical design. Attempting to construct a proof often reveals edge cases or logical inconsistencies that were not apparent during informal derivation. Second, the proof script itself becomes a form of documentation, providing a human-readable, machine-verifiable description of the formula's intended behavior. Third, and most importantly, the verified specification can be used to generate or check code implementations. There are two primary approaches here. The first is **formal code extraction**, where the proof assistant is used to mechanically generate executable code in a target language like Rust or Lua from the verified mathematical function [54]. This drastically reduces the risk of implementation bugs, as the gap between the abstract specification and the concrete code is minimized. The second approach is **property-based testing**, where the extracted or manually written Rust/Lua functions are treated as candidates for implementation, and a battery of automated tests is run to verify that they satisfy the same properties (monotonicity, boundedness) proven in the theorem prover [55]. Any function that fails these property tests is considered incorrect.

Integrating formal verification into the CI/CD pipeline further strengthens this process. A CI job could be added to run the Lean/Coq proofs whenever a change is made to a timing formula or its parameters. The pipeline would fail if any proof obligation cannot be discharged, forcing the developer to correct the specification or the proof before proceeding. This creates a hard gate that ensures the mathematical soundness of the timing logic is continuously maintained. Furthermore, the results of these proofs can inform the linting rules applied to AI-generated code. Linters like `aifilelint` can be enhanced to understand the proven properties of the timing helpers, allowing them to catch potential misuse or incorrect assumptions in the Lua scripts that call these functions [30]. For example, a linter could warn if a script attempts to pass an out-of-bounds value

to a helper, even if the value is valid according to the JSON schema, because the linter knows from the formal proof that the helper's behavior is only guaranteed within a specific sub-domain. By combining the empirical insights from telemetry with the mathematical rigor of formal methods, the Horror$Place ecosystem can achieve a much higher degree of confidence in the correctness, safety, and predictability of its systemic timing mechanics.

Implementation Roadmap and Code Quality Enforcement

The enhancement of Horror$Place's systemic timing mechanics is a complex undertaking that requires a phased, methodical approach to ensure stability and maintainability. The proposed implementation roadmap is designed to progressively build upon a solid foundation of contracts and schemas, moving from static validation to dynamic empirical analysis and finally to mathematical proof. This phased progression minimizes risk and ensures that each new capability is properly integrated and enforced by the existing toolchain. The entire process is underpinned by a strict commitment to the contract-first doctrine, with all code and data artifacts being subject to rigorous validation to maintain high code quality [66].

The first phase, **Contract & Schema Foundation**, focuses on establishing the necessary metadata and validation rules before any implementation changes are made. The primary action is to extend the central `invariants_v1.json` schema in the `Horror.Place` repository with any new, fundamental invariants and, more critically, to create the new `mechanic_contract_timing_params_v1.json` schema in `HororPlace-Constellation-Contracts` [5]. This timing parameters schema will define the structure and constraints for all coefficients used in the new formulas, including ranges for weights, thresholds, and normalization bounds [69]. The success metric for this phase is the successful integration of a CI workflow that validates all mechanic contracts against this new schema. Any contract file that contains invalid parameters (e.g., a weight outside its defined range) will be rejected by the CI pipeline, enforcing a baseline level of correctness at the earliest stage of the development process [29].

The second phase, **Implementation & Empirical Validation**, moves from static schemas to dynamic, runtime behavior. The core task is to refactor the Lua timing helpers in `engine/systemic_timing.lua` to be formula-driven, ensuring they source all

coefficients exclusively from the newly created timing parameters schema rather than from hardcoded values [19]. Concurrently, these helpers must be instrumented to emit detailed telemetry records conforming to the `systemic-timing-formula-run.v1` schema [25]. The goal of this phase is to enable the empirical validation of the formulas. This is achieved by using analysis tools, such as the `systemic_timing_analysis.ipynb` notebook, to mine the telemetry data. Analysts can correlate helper decisions (e.g., the computed `p_downgrade`) with the resulting changes in player metrics (ΔUEC, ΔARR) to tune the formula parameters and identify configurations that produce the desired player experience [72] [73]. The success metric is the identification of one or more parameter sets that demonstrably improve UEC without collapsing ARR, providing concrete evidence that the timing mechanics are functioning as intended.

The third phase, **Formal Verification**, introduces mathematical rigor to complement empirical testing. The objective is to encode the timing formulas in a proof assistant like Lean and construct machine-checked proofs for their key properties: monotonicity and boundedness [54]. This phase also involves exploring code generation or verification techniques to connect the formal specification to the actual Rust and Lua implementations. The success criterion is the complete discharge of all proof obligations within the theorem prover, accompanied by the generation and integration of verified code or the establishment of a CI gate that runs property-based tests against the hand-written implementations. This provides a higher level of assurance that the timing logic is free from logical flaws and will behave predictably across all scenarios.

The fourth and final phase, **Cross-Engine Hardening**, ensures the engine-agnostic promise is fulfilled. This involves finalizing the `engine_api_surface_v1.json` schema, which defines the canonical API that all engine ports must implement [5]. CI pipelines for each engine (Rust, C++, C#) are then updated with conformance checks that validate the engine's `H.*` stubs and timing helper implementations against this schema. The success metric is that all engine implementations pass identical runtime conformance tests, proving that the invariant-aware policy engine behaves identically across all supported platforms [29].

Throughout this entire roadmap, a suite of tools and practices enforces code and document quality. CHATDIRECTOR's multi-layer validation pipeline is expanded to include checks for the new timing parameters and narrowing rules [38]. Pre-commit hooks and authoring scripts integrate Lua contract linters that provide instant feedback on band violations and other contract breaches [30]. Finally, lints like `aifilelint` are hardened to specifically detect the misuse of AI metadata and other artifacts, ensuring that the

generated code remains clean and focused on the core logic. By following this comprehensive, phased approach, the Horror$Place ecosystem can systematically and safely elevate its systemic timing mechanics to a new level of robustness, predictability, and data-driven refinement.

Reference

1. (PDF) Electroacoustic and Computer Music Timelines - Academia.edu https://www.academia.edu/38276433/Electroacoustic_and_Computer_Music_Timelines
2. Common TCP/UDP Port Numbers Guide | PDF - Scribd https://www.scribd.com/document/500229134/TCP-and-UDP-Port-Numbers-Most-Common-Port-Numbers
3. 5 - CodaLab Worksheets https://worksheets.codalab.org/rest/bundles/0xd74f36104e7244e8ad99022123e78884/contents/blob/frequent-classes
4. 333333 23135851162 the 13151942776 of 12997637966 ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt
5. A Full-Stack Benchmark for Privacy Leakage in Multi-Agent LLM ... https://arxiv.org/html/2602.11510v2
6. Gartner's Essential Skills for Software Engineers | Aaron Lord posted ... https://www.linkedin.com/posts/aaronlord_i-was-reviewing-cisas-secure-by-design-material-activity-7432464060056567808-MZCR
7. BIM–FM Interoperability Through Open Standards: A Critical ... - MDPI https://www.mdpi.com/2673-8945/5/3/74
8. [PDF] Junos Telemetry User Guide - Juniper Networks https://www.juniper.net/documentation/us/en/software/junos/interfaces-telemetry/interfaces-telemetry.pdf
9. Ensuring the Maintainability and Supportability of "Vibe-Coded ... https://www.researchgate.net/publication/391491700_Ensuring_the_Maintainability_and_Supportability_of_Vibe-Coded_Software_Systems_A_Framework_for_Bridging_Intuition_and_Engineering_Rigor
10. 翻译[8]-rust第三方库整合包- qsBye - 博客园 https://www.cnblogs.com/qsbye/p/19150896
11. How to deserialize a JSON encoded DateTime with a specified ... https://stackoverflow.com/questions/78838933/how-to-deserialize-a-json-encoded-datetime-with-a-specified-timezone-in-rust-wit

12. 有问题，就会有答案 - 知乎 https://www.zhihu.com/en/article/1908582286041014708
13. 333333 23135851162 the 13151942776 of 12997637966 https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt
14. Google 1-gram - CMU School of Computer Science https://www.cs.cmu.edu/~roni/11761/2017_fall_assignments/hw3_stats_google_1gram.txt
15. Old Irish For Beginners | PDF - Scribd https://www.scribd.com/doc/123082009/Old-Irish-for-Beginners
16. Elevating Intelligence via Efficient Model and Tool Orchestration https://arxiv.org/html/2511.21689v1
17. The Complete Full-Stack Developer Roadmap for 2026 https://dev.to/thebitforge/the-complete-full-stack-developer-roadmap-for-2026-2i0j
18. [PDF] The 14th International Joint Conference on Natural Language ... https://aclanthology.org/2025.ijcnlp-srw.pdf
19. Lua scripting in FRR: A flexible BGP policy tool - LinkedIn https://www.linkedin.com/posts/yanfilyurin_github-yanf787frrlua-activity-7394995642026213376-uagM
20. Trustworthy AI Posture (TAIP): A Framework for Continuous AI ... https://arxiv.org/html/2603.03340v1
21. Towards a Reference Architecture for Machine Learning Operations https://www.mdpi.com/2073-431X/15/4/218
22. [PDF] D7.10 – Standardization Plan and Status Report v2 https://ec.europa.eu/research/participants/documents/downloadPublic?documentIds=080166e50d2df96f&appId=PPGMS
23. A System Specification for Securing AI-Driven Actions at Runtime https://www.researchgate.net/publication/400661128_Autonomous_Action_Runtime_ManagementAARMA_System_Specification_for_Securing_AI-Driven_Actions_at_Runtime
24. [PDF] Contents - Broadcom TechDocs https://techdocs.broadcom.com/content/dam/broadcom/techdocs/us/en/pdf/vmware-tanzu/platform/tanzu-rabbitmq-tanzu-platform/10-1/rabbitmq-tp/rabbitmq-tp.pdf
25. Interfaces Telemetry | PDF | Computer Network - Scribd https://www.scribd.com/document/792935004/Interfaces-Telemetry
26. Experts for Nodejs-Mobile-React-Native Plugins Readme - Linknovate https://www.linknovate.com/search/?query=nodejs-mobile-react-native%20plugins%20readme
27. Lua json schema validator - Stack Overflow https://stackoverflow.com/questions/56401882/lua-json-schema-validator

28. Requested packages, organized by age - Debian https://www.debian.org/devel/wnpp/requested_byage
29. [PDF] Computer Science Curricula 2023 - CS2023 https://csed.acm.org/wp-content/uploads/2023/03/Version-Beta-v2.pdf
30. 平台和工具 - 架构师 https://architect.pub/book/export/html/36
31. Steam 创意工坊::ruzgiş rp https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=3057925259
32. [PDF] C/2024/2816 - European Union https://eur-lex.europa.eu/legal-content/RO/TXT/PDF/?uri=OJ:C_202402816
33. Ted Timmins - Senior Director, Design - Fortnite at Epic Games https://www.linkedin.com/in/tedtimmins
34. (PDF) MI9 -- Agent Intelligence Protocol: Runtime Governance for ... https://www.researchgate.net/publication/394361987_MI9_--_Agent_Intelligence_Protocol_Runtime_Governance_for_Agentic_AI_Systems
35. 机器学习2026_4_7 - arXiv每日学术速递 http://www.arxivdaily.com/thread/78520
36. Arxiv今日论文| 2026-02-25 - 闲记算法 http://lonepatient.top/2026/02/25/arxiv_papers_2026-02-25.html
37. Architecting a Rovo AI Teammate: From AI 'Magic' to Production ... https://www.atlassian.com/blog/it-teams/architecting-a-rovo-ai-teammate-from-ai-magic-to-production-ready-forge-code
38. I'm Building My Own Coding Agent Harness (And It's Pretty Cool) https://dev.to/composiodev/im-building-my-own-coding-agent-harness-and-its-pretty-cool-1lpf
39. AI Agents' Hidden Reality: Compounding Tech Debt at Machine Pace https://www.linkedin.com/posts/nihitnirmal_everywhere-i-look-teams-are-celebrating-activity-7437380661440561152-tYYb
40. [PDF] bedrock-agentcore-dg.pdf - AWS Documentation https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/bedrock-agentcore-dg.pdf
41. [PDF] IWSDS 2026 The 16th International Workshop on Spoken Dialogue ... https://aclanthology.org/2026.iwsds-1.pdf
42. Software Portability for Computer Algebra - arXiv https://arxiv.org/html/2506.01864v1
43. [PDF] ACL 2018 Neural Machine Translation and Generation Proceedings ... https://aclanthology.org/W18-27.pdf
44. Runtime Verification - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-030-60508-7.pdf
45. [PDF] Inside the CENTEL Intelligence Adjacency Lab - Zenodo https://zenodo.org/records/19133974/files/

%F0%9F%9F%AACENTEL%E2%84%A2%EF%B8%8F_Dreams,
%20Behavioral%20Grit,%20Lies,
%20Signal%20Purity%20&%20Signal%20Drift_%20Inside%20the%20CENTEL%20Intelligence%20Adjacency%20Lab%20(Final%20v2).pdf?download=1

46. [PDF] Proceedings of the 4th Workshop on e-Commerce and NLP https://aclanthology.org/2021.ecnlp-1.pdf
47. Design Thinking with Artificial Intelligence - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-032-10543-1.pdf
48. Uncovering 'Weak signals' through patient journey mapping to ... https://link.springer.com/article/10.1186/s12913-026-14037-9
49. Non-Canonical Syntax in Register-Based Varieties of English (Part II) https://www.cambridge.org/core/books/noncanonical-english-syntax/noncanonical-syntax-in-registerbased-varieties-of-english/BEC83DF0A4515822C9B027097DD6B2FA
50. Online Engineering | PDF | Internet Of Things - Scribd https://www.scribd.com/document/739508613/Online-Engineering
51. Export your star's repository list - starList - Gitea: Git with a cup of tea https://gitea.com/hiifong/starList
52. Bold Custom Pricing for Shopify Apps | PDF - Scribd https://www.scribd.com/document/887297848/Shopify-App
53. https://dq-content.s3.amazonaws.com/356/hacker_new... https://dq-content.s3.amazonaws.com/356/hacker_news.csv
54. Stop Parsing LLMs with Regex: Build Production-Ready AI ... https://dev.to/dthompsondev/llm-structured-json-building-production-ready-ai-features-with-schema-enforced-outputs-4j2j
55. Easily enforcing valid JSON schema following - API https://community.openai.com/t/feature-request-function-calling-easily-enforcing-valid-json-schema-following/263515
56. Arxiv今日论文| 2026-02-16 - 闲记算法 http://lonepatient.top/2026/02/16/arxiv_papers_2026-02-16
57. Interoperable Open Specifications Framework for the ... - MDPI https://www.mdpi.com/1424-8220/20/8/2402
58. [PDF] Proceedings of the The First Workshop on LLM Security (LLMSEC) https://aclanthology.org/2025.llmsec-1.pdf
59. Experts for Nice Vr-Native Reload Style - Linknovate https://www.linknovate.com/search/?query=nice%20vr-native%20reload%20style
60. Dictionary of Acronyms and Technical Abbreviations - Academia.edu https://www.academia.edu/64999604/Dictionary_of_Acronyms_and_Technical_Abbreviations

61. Comprehensive Glossary of Telecom | PDF - Scribd https://it.scribd.com/doc/266101671/Comprehensive-Glossary-of-Telecom
62. Innovative Computing and Communications - Springer Nature https://link.springer.com/content/pdf/10.1007/978-981-96-6906-6.pdf
63. (PDF) A STATISTICAL GENERALIZED PROGRAMMING ALGORITHM https://www.academia.edu/3047123/A_STATISTICAL_GENERALIZED_PROGRAMMING_ALGORITHM
64. Advances in Data-Driven Computing and Intelligent Systems https://link.springer.com/content/pdf/10.1007/978-981-96-5370-6.pdf
65. From Logic Monopoly to Social Contract: Separation of Power and ... https://arxiv.org/html/2603.25100v1
66. (PDF) Quality Analysis and Management for Construction Document ... https://www.researchgate.net/publication/398030566_Quality_Analysis_and_Management_for_Construction_Document_Production-r24
67. Lua - What is the most optimized method of a Lua function? https://stackoverflow.com/questions/28311020/lua-what-is-the-most-optimized-method-of-a-lua-function
68. 1 - Amazon S3 https://s3.eu-central-1.amazonaws.com/deepset.ai-farm-models/0.4.1/glove-english-uncased-6B/vocab.txt
69. How to enforce restrictions in Json Schema https://stackoverflow.com/questions/33904298/how-to-enforce-restrictions-in-json-schema
70. (PDF) The Open -On the Metaphysics of the Semantic Field A fissure ... https://www.researchgate.net/publication/395868305_The_Open_-On_the_Metaphysics_of_the_Semantic_Field_A_fissure_through_which_meaning_breathes
71. (PDF) DIGITAL TWIN ROADWAYS: INTEGRATING BIM, IOT AND AI ... https://www.researchgate.net/publication/395366176_DIGITAL_TWIN_ROADWAYS_INTEGRATING_BIM_IOT_AND_AI_FOR_INTELLIGENT_INFRASTRUCTURE_MANAGEMENT
72. Arxiv今日论文| 2026-03-12 - 闲记算法 http://lonepatient.top/2026/03/12/arxiv_papers_2026-03-12.html
73. 机器学习2026_1_13 - arXiv每日学术速递 http://arxivdaily.com/thread/75632
74. A Thematic Analysis Using an Action-Block Framework - arXiv https://arxiv.org/html/2603.18398
75. Artificial Intelligence Applications and Innovations - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-030-49186-4.pdf

76. [PDF] Huawei CloudEngine S6750-S Series Switches Datasheet https://e.huawei.com/marketingcloud/pep/asset/20000001/Material/7fbd3d5ba8754b7989d7bc66b0be786e/M3T1A590N1089494133801136181/Huawei%20CloudEngine%20S6750-S%20Series%20Switches%20Datasheet.pdf
77. RFC 5197: On the Applicability of Various ... - ACM Digital Library https://dl.acm.org/doi/pdf/10.17487/RFC5197
78. Cisco Industrial Ethernet 1000 Series Switches Data Sheet https://www.cisco.com/c/en/us/products/collateral/switches/industrial-ethernet-1000-series-switches/datasheet-c78-737277.html